

Agentic AI for Telemetry Analysis on AROL Capping Machines



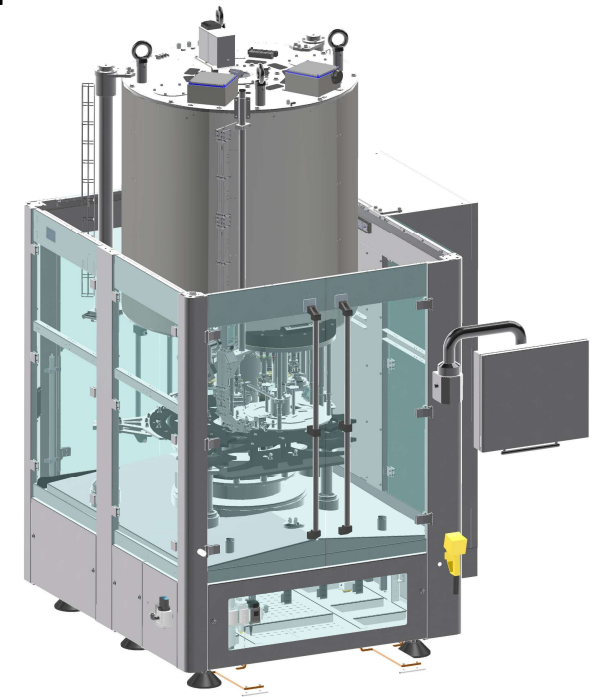
CONTEXT

AROL capping machines and related subsystems can expose a variety of automation and model-specific measurements useful for monitoring performance, diagnosing anomalies, and enabling predictive/preventive maintenance.

Examples of measurable data include:

- machine status and alarms
- production rate and downtime reasons
- OEE-related indicators
- energy metering
- **torque/closure diagnostics** (e.g., average torque, closure status)

For this project proposal we will concentrate only on **torque/closure diagnostics**

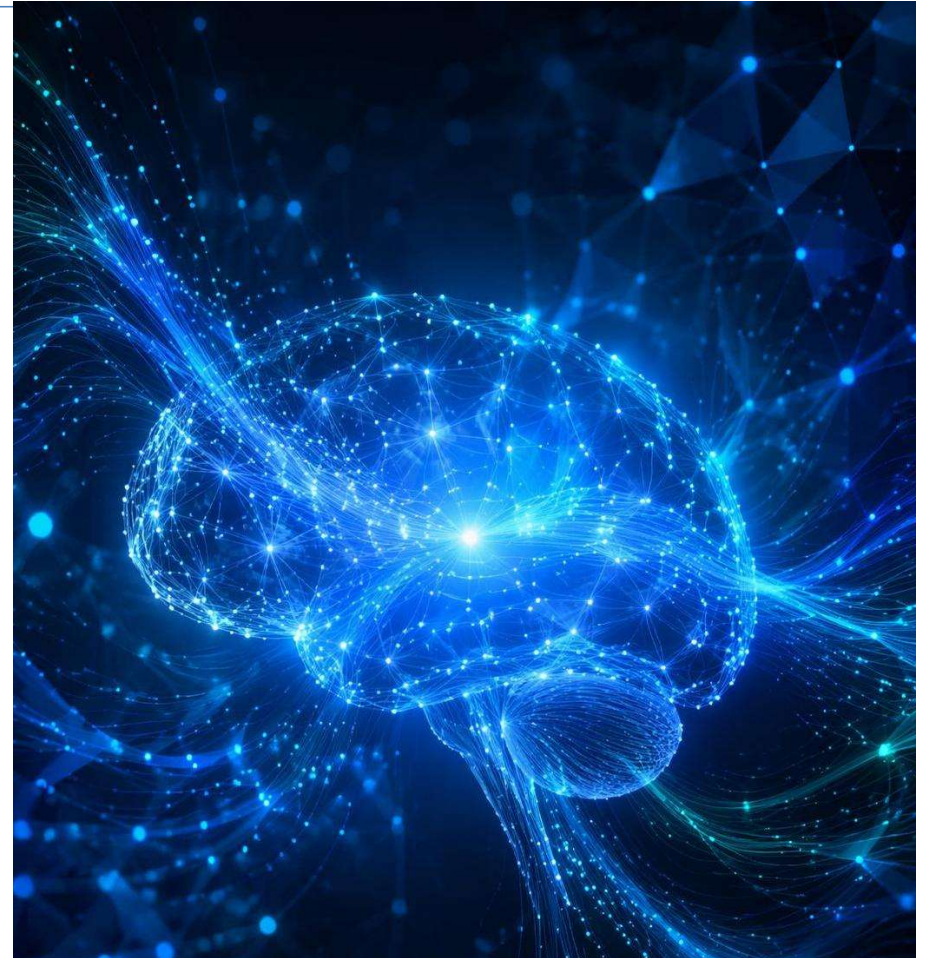


OBJECTIVE

Design and implement an Agentic AI application

The application must implement:

1. **Ingests and organizes telemetry/measurement datasets** coming from AROL capping machines (provided as a small number of curated “data pools”)
2. Provides a **BOT interface** (CLI, web, or lightweight UI) that can **generate analysis reports on demand** (e.g., anomaly summaries, trends, correlations, basic KPIs), selecting appropriate analysis steps autonomously
3. Produces **repeatable, explainable outputs** (reports + plots/tables where relevant), suitable for technical users in R&D/Service.



DETAIL ABOUT PROVIDED MEASUREMENTS

Your solution must be designed to work on datasets that may include, for instance:

Capping diagnostic data:

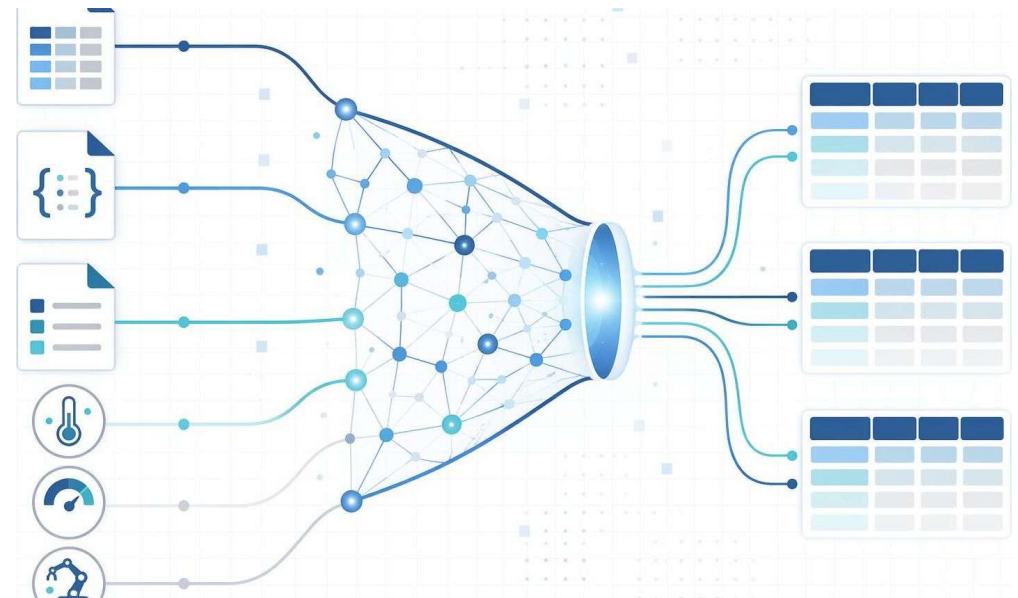
- Timestamp
- Per-head closure torque
- Per-head closure status.

timestamp	H01 Count	H01 AppTorque	H01 Status	H02 Count	H02 AppTorque	H02 Status	...	H48 AppTorque	H48 Status
2026-05-20T02:26:12Z	23553	2.54	0	23499	2.51	0		2.48	0
2026-05-20T02:26:13Z	23553	2.54	0	23500	1.83	64		2.48	0
2026-05-20T02:26:14Z	23554	2.53	0	23500	1.83	64		2.52	0
2026-05-20T02:26:15Z	23554	2.53	0	23500	1.83	64		2.52	0
2026-05-20T02:26:16Z	23554	2.53	0	23501	2.48	0		2.52	0
2026-05-20T02:26:17Z	23555	2.49	0	23501	2.48	0		2.45	0

CORE TASKS

Data ingestion & normalization

- Implement a pipeline that loads one or more dataset pools (e.g., CSV/JSON, Parquet) and normalizes them into a consistent internal schema.
- Provide validation checks (missing values, timestamp consistency, units metadata).
- Detects which heads have “closed” by comparing counters respect previous records.
- For each detected closure head, collects closure-related fields (torque, status)
- Computes a capping speed (pieces/hour) using an incremental average.
- Compute the timestamp for each closure



CORE TASKS

Baseline analytics layer

Implement a first set of deterministic analyses (so the AI agent has reliable tools to call):

- Trend analysis (moving averages, drift detection) on Torque values
- Simple anomaly detection (thresholds, statistical deviations)
- Correlation checks among selected signals (e.g. performance of head 1 vs head 5)
Use the telemetry categories above as the expected signal families.
- Identify machine idle state based on "No Load" for a sustained period for every head

Closure Status	Reject Signal	Error Description
0	NO	Closure OK
2	NO	No Load
3	YES	Failing to reach the first torque threshold (SlowTorque)
4	NO	No Closure
5	YES	Failing to reach the final torque (ClosureTorque)
8	NO	No InTorque
9	YES	Closure Head raises before the TimeInTorque time was elapsed
16	NO	No CapTurns
17	YES	The cap is closed but with less degrees than CapTurns
32	NO	Following Error
33	YES	Tracking error between the real position and the controlled position of the head
64	NO	Bad Closure
65	YES	ClosureTorque reached but cap is still rotating when head raises



Agentic AI orchestration (“report agent”)

Build an **agent** that can:

- Interpret a user request (e.g., “Generate a weekly anomaly report for Line X” / “Explain why downtime increased”)
- Decide which tools to run (WP2 functions), in which order
- Produce a structured report with: *goal* → *data used* → *analyses executed* → *findings* → *confidence/limits* → *next checks*

This matches the proposed direction of a BOT that generates one or more analysis reports from shared data pools.

CORE TASKS

WP4 - BOT interface (System & Device Programming angle)

Choose one of the following, prioritizing robustness and deployability:

- **CLI tool** (preferred for fast iteration): commands like report anomalies, report drift, report kpi
- **Minimal web UI** (optional) with report download
- **Local service** exposing REST endpoints for report generation

CORE TASKS

Engineering quality & reproducibility

- Configuration-driven datasets (no hard-coded paths)
- Logging, error handling, test cases on at least one dataset pool
- Documented build/run instructions

PREREQUISITES REQUIRED TO THE STUDENTS

This project is **not** a “plug-and-play chatbot”: it requires real engineering effort on data, tooling, and reliability.

Candidates should have:

- **Strong programming autonomy** (ability to design modules, debug, and deliver without step-by-step supervision)
- Solid skills in **Python or another systems-capable language** (plus scripting, packaging, CLI tools)
- Confidence with **data handling** (CSV/JSON parsing, timestamps, basic statistics)
- Basic knowledge of **software architecture** (layers, interfaces, testability)
- A minimum ability to read technical specs and reason about industrial signals (alarms/states/counters/diagnostics), such as the telemetry families described in AROL material.



DELIVERABLES

What you must submit:

- Source code repository (clean structure, reproducible run)
- Report templates (e.g., Markdown/HTML/PDF export)
- Sample generated reports
- Technical documentation:
 - architecture,
 - data schema,
 - analytics methods,
 - agent decision flow
- Demo: one end-to-end run that loads a dataset pool and generates at least two different report types
- Short final presentation (10–15 slides)

EVALUATION CRITERIA

- Correctness and robustness of ingestion/normalization
- Quality and usefulness of analytics outputs
 - signal handling
 - drift/anomaly evidence
- Agentic orchestration quality
 - clear tool-use flow
 - explainable outputs
 - graceful failures
- Software engineering quality (tests, structure, documentation, reproducibility)
- Demo clarity and technical communication

EXAMPLES OF QUERIES (1/3)

Basic data exploration queries

These help the user understand the dataset after preprocessing.

- "How many capping operations were performed in the selected month?"
- "How many closure events were performed by each head?"
- "Show me the time range covered by the dataset."
- "Are there any missing or invalid torque values?"

Quality and success-rate queries

These directly match your functional requirement about **successful capping operations**.

- "What percentage of capping operations were successful?"
- "How many closures ended with a positive outcome?"
- "How many failed capping operations were recorded?"
- "What is the success rate per capping head?"
- "Which head shows the lowest success rate?"

Example expected answer:

- "92.4% of the capping operations resulted in a positive outcome over the analyzed period."

Torque-related analytical queries

Used to assess process stability and mechanical behavior.

- "What is the average closing torque for successful capping operations?"
- "Show the torque distribution for all successful closures."
- "Are there torque values outside the expected operating range of?"
- "Compare the average torque of successful vs failed closures."
- "Which head shows the highest torque variability?"

EXAMPLES OF QUERIES (2/3)

Time-based and trend queries

These exploit timestamps after event reconstruction.

- "How did the capping success rate evolve over time?"
- "Show a daily breakdown of successful vs failed closures."
- "Are there specific time intervals with abnormal failure rates?"
- "Did the average torque change over the observed month?"
- "Is there a correlation between time of day and failure probability?"

Filtering and conditional queries

Queries combining **status, torque, and counters**.

- "Show only capping operations with a positive outcome."
- "List all failed capping events with torque below threshold."
- "How many closures had torque above X Nm?"
- "Show all capping events for head 3 with failed outcome."
- "Count successful closures after removing all duplicated entries."

Diagnostic and comparative queries

More advanced, but still feasible with classical analytics or simple AI reasoning.

- "Which capping head behaves differently from the others?"
- "Is there a head with an unusual number of failed closures?"
- "Compare performance between head 1 and head 2."
- "Which head contributes most to overall failures?"
- "Does higher torque correlate with higher success rate?"

EXAMPLES OF QUERIES (3/3)

Explanation-oriented queries (agentic behavior)

These test the **reasoning and explanation capability** of the bot.

- "Why is the overall success rate lower on certain days?"
- "Explain why head 4 has more failed closures."
- "Summarize the main issues observed in the capping process."
- "Which signals should be monitored more closely?"
- "Generate a short report on capping quality for this dataset."

Visualization-oriented queries

These link directly to the **student-designed visualization part**.

- "Plot the closing torque over time for successful closures."
- "Show a histogram of closing torque values."
- "Create a chart showing success rate per head."
- "Visualize failed closures over time."
- "Generate a dashboard summary of capping performance."

Meta / system queries

Useful for usability and transparency in a thesis project.

- "What preprocessing steps were applied to the raw data?"
- "How were duplicated closures detected and removed?"
- "Which assumptions were made during data cleaning?"
- "What features are used to classify a successful closure?"

EXPECTED OUTPUTS (1)

Overall process quality

User query:

"What percentage of capping operations were successful in the selected period?"

Expected system behavior

- Use the **cleaned, event-based dataset**
- Count only **unique closure events**
- Classify events by *closure status*

Example output (illustrative)

Total closure events analyzed: 1,248,320

Successful closures: 1,164,870

Failed closures: 83,450

Overall success rate: 93.3%

Explanation provided by the bot

"After removing duplicated records and reconstructing individual closure events, 93.3% of the capping operations resulted in a successful outcome over the analyzed one-month period."

(Demonstrates: preprocessing → aggregation → KPI generation)

EXPECTED OUTPUTS (2)

Head-level performance comparison

User query:

"What is the success rate for each capping head?"

Expected system behavior

- Group events by **head ID**
- Compute success rate per head
- Highlight deviations

Example output (table format)

Head ID	Total Closures	Successful	Success Rate
Head 1	208,430	198,900	95.4%
Head 2	207,980	194,120	93.3%
Head 3	207,540	188,760	90.9%
Head 4	208,190	199,840	96.0%
Head 5	207,180	183,250	88.5%

Explanation provided by the bot

"Head 5 shows a significantly lower success rate compared to the others and may require further investigation."

EXPECTED OUTPUTS (3)

Torque statistics for successful closures

User query

"What is the average closing torque for successful capping operations?"

Expected system behavior

- Filter **status** = **successful**
- Exclude duplicated records
- Aggregate torque values

Example output (illustrative)

Successful closure events analyzed: 1,164,870

Average torque: 2.41 Nm

Minimum torque: 1.85 Nm

Maximum torque: 3.12 Nm

Standard deviation: 0.22 Nm

Explanation provided by the bot

"The torque values show a stable distribution around the nominal operating range, suggesting correct overall mechanical behavior during successful closures."

(Demonstrates: filtering + statistical summarization)

Thank you for your kind attention

arol.com

follow us on



BACKUP SLIDES
